

上节回顾

- 进程同步的任务：P47
- 生产者--消费者问题
- 临界资源：增加锁，wait(s)和signal(s)操作
 - wait(s)，检查锁状态，上锁；signal(s)，开锁
- 同步机制的规则
 - 空闲让进、忙则等待、有限等待、让权等待
- 整型信号量：只有一个整数变量记录可用资源数量
 - 不满足让权等待的规则
- 记录型信号量：请求1种资源，数量1个
 - 1个整数变量value和1个阻塞队列L
 - $value > 0$ ，表示资源可用数量， $value < 0$ ，表示阻塞进程个数
- AND型信号量：请求n种资源，每种数量1个
 - 多个信号量，资源要么全部分配，要么一个都不分配
- 信号量集：请求n种资源，每种数量n个
 - 通用的信号量表示方式，其他类型信号量都是它的特例
 - 多个信号量，每种资源都带下限值

2.3 进程同步

2.3.3 信号量的应用

- 利用信号量实现单个资源互斥共享

```
Var mutex : semaphore : = 1;
```

```
begin
```

```
parbegin
```

```
    process 1: begin
```

```
        repeat
```

```
            wait(mutex);
```

```
            critical section
```

```
            signal(mutex);
```

```
            remainder section
```

```
        until false;
```

```
        end
```

```
parend
```

```
    process 2: begin
```

```
        repeat
```

```
            wait(mutex);
```

```
            critical section
```

```
            signal(mutex);
```

```
            remainder section
```

```
        until false;
```

```
        end
```

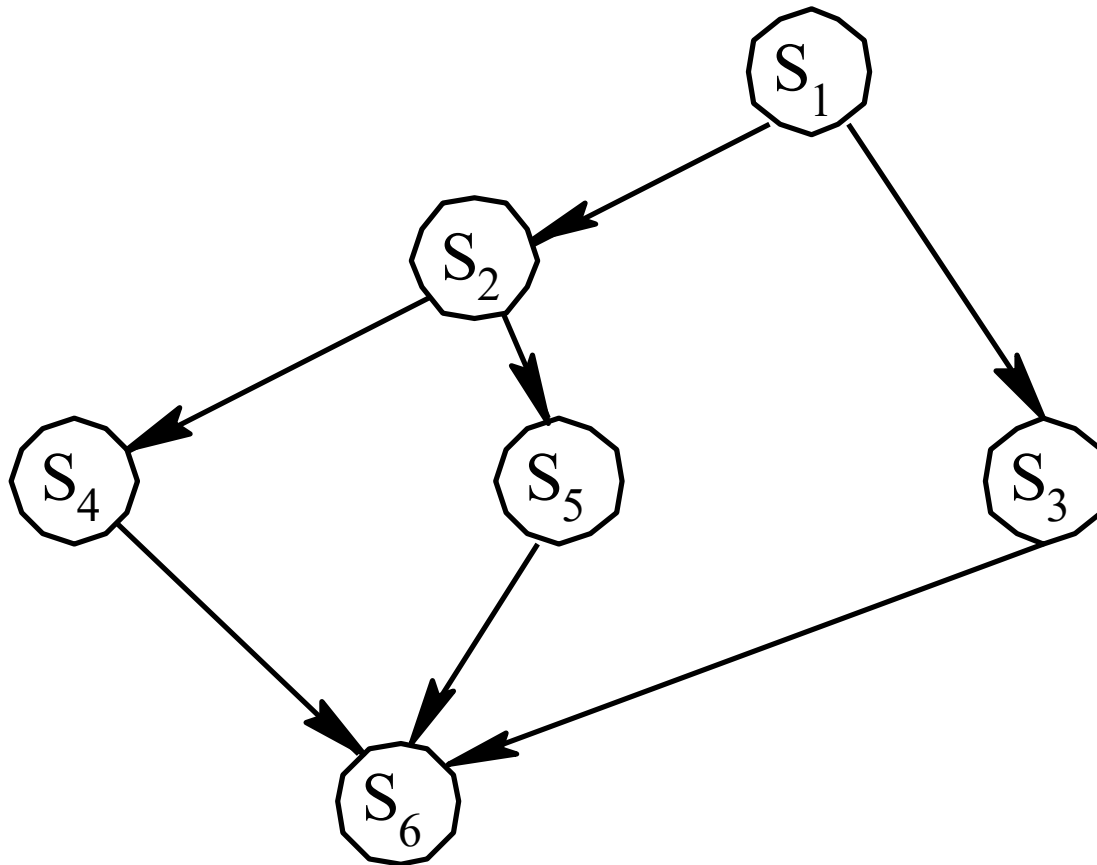
2.3 进程同步

■ 信号量机制的特性

- ❑ **wait(S)和signal(S)必须成对出现，但可以不**
在同一个进程中
- ❑ 缺少**wait(S)**不能保证资源互斥
- ❑ 缺少**signal(S)**可能使资源永远不能释放
- ❑ 不存在“忙等”问题

2.3 进程同步

- 利用信号量实现前趋关系



2.3 进程同步

■ 利用信号量实现前趋关系

```
Var a,b,c,d,e,f,g; semaphore := 0,0,0,0,0,0,0;
```

```
begin
```

```
  parbegin
```

```
    begin S1; signal(a); signal(b); end;
```

```
    begin wait(a); S2; signal(c); signal(d); end;
```

```
    begin wait(b); S3; signal(e); end;
```

```
    begin wait(c); S4; signal(f); end;
```

```
    begin wait(d); S5; signal(g); end;
```

```
    begin wait(e); wait(f); wait(g); S6; end;
```

```
  parend
```

```
end
```

2.3 进程同步

2.3.4 管程

■ 信号量机制的不足

- ❑ 临界区分布在不同进程中，不便于管理与控制
- ❑ 很多临界区操作是相同的，代码重复编写，耗费工作量
- ❑ 程序中加入**wait**和**signal**操作，结构不清晰，可读性差
- ❑ 程序中若出现错误，常因为阻塞而不便于调试跟踪
- ❑ 程序中若**wait**和**signal**编写错误，难于预测而且易引发死锁

2.3 进程同步

- 管程的定义：P55

- 管程的组成

- ① 局部于管程的共享变量说明
- ② 对该数据结构进行操作的一组过程
- ③ 对局部于管程的数据设置初始值的语句
- ④ 管程具有一个名字

- 管程的优点：

- 管程实质是采用面向对象技术的一种封装
- 使得进程访问临界资源时非常方便
- 程序编写简洁，结构清晰
- 易于查错，而且大大降低了死锁发生的可能性

2.4 经典进程的同步问题

2.4.1 生产者—消费者问题

- 生产者—消费者问题是相互合作的进程关系的一种抽象，该问题有很大的代表性及实用价值
 - 输入时，输入进程是生产者，计算进程是消费者；
 - 输出时，计算进程是生产者，打印进程是消费者

2.4 经典进程的同步问题

■ 未加信号量的生产者—消费者程序

producer: repeat

...

produce an item in nextp;

...

while counter=n do no-op;

buffer [in] · : = nextp;

in : = in+1 mod n;

counter : = counter+1;

until false;

consumer: repeat

while counter=0 do no-op;

nextc : = buffer [out] ;

out : = (out+1) mod n;

counter : = counter-1;

consumer the item in nextc;

until false;

2.4 经典进程的同步问题

- 采用互斥型信号量解决生产者—消费者问题

Var mutex : semaphore : = 1;

producer: repeat

...

wait (mutex);

counter : = counter+1;

signal (mutex);

...

consumer: repeat

...

wait (mutex);

counter: = counter-1;

signal (mutex);

...

- 未实现两种限制情况

- ☐ 缓冲池全满则生产者阻塞

- ☐ 缓冲池全空则消费者阻塞

- ☐ 原有的**counter**是一个整数变量，非信号量，阻塞队列不能挂靠在**counter**对应的队列中

2.4 经典进程的同步问题

1. 利用记录型信号量解决生产者—消费者问题

- 具有 n 个缓冲区公用缓冲池
- 互斥信号量**mutex**实现诸进程对缓冲池的互斥使用
- 信号量**empty**表示缓冲池中空缓冲区数量。
- 信号量**full**表示缓冲池中满缓冲区的数量。
- **mutex**为公用信号量，**full**与**empty**为私用信号量
- 缓冲池未滿，生产者可以将消息送入缓冲池
- 缓冲池未空，消费者可以从缓冲池中取走消息

2.4 经典进程的同步问题

- 加入记录型信号量的生产者—消费者程序

Var mutex, empty, full:semaphore := 1,n,0;

buffer:array [0, ..., n-1] of item;

in, out: integer := 0, 0;

- **mutex=1**，用于互斥
- **empty=n**，缓冲池初始化是全空
- **full=0**，若消费者进程先启动，则进入阻塞状态等待生产者

2.4 经典进程的同步问题

- 加入记录型信号量的生产者—消费者程序

producer:begin

repeat

producer an item nextp;

wait(empty);

wait(mutex);

buffer(in):= nextp;

in:= (in+1) mod n;

signal(mutex);

signal(full);

until false;

nextc;

consumer:begin

repeat

...

wait(full);

wait(mutex);

nextc:= buffer(out);

out:= (out+1) mod n;

signal(mutex);

signal(empty);

consumer the item in

...

2.4 经典进程的同步问题

- 互斥（公用）信号量，在单个程序中必须成对地出现
 - 例如实现互斥的`wait(mutex)`和`signal(mutex)`
- 资源（私用）信号量`empty`和`full`的`wait`和`signal`操作，同样需要成对地出现，但它们分别处于不同的程序中。
 - 例如，`wait(empty)`在计算进程中，而`signal(empty)`则在打印进程中，计算进程若因执行`wait(empty)`而阻塞，则以后将由打印进程将它唤醒
- 每个程序中的多个`wait`操作顺序不能颠倒。应先执行对资源信号量的`wait`操作，然后再执行对互斥信号量的`wait`操作，否则可能引起进程死锁。——“先私后公”

2.4 经典进程的同步问题

■ 死锁的发生：先公后私

- 生产者和消费者进程都交换了wait操作的次序
- $\text{mutex}=1, \text{full}=0, \text{empty}=n$
- ① C: $\text{wait}(\text{mutex}) \rightarrow \text{mutex}-1=0$, 通过往下执行
- ② C: $\text{wait}(\text{full}) \rightarrow \text{full}-1=-1<0$, 消费者进程被阻塞
- ③ P: $\text{wait}(\text{mutex}) \rightarrow \text{mutex}-1=-1<0$, 生产者进程被阻塞
- 这时，生产者和消费者两进程分别阻塞在等待**mutex**和**full**信号量的队列，没有进程可以向前推进，系统进入死锁状态

2.4 经典进程的同步问题

- 假设缓冲池有3个缓冲区，当前执行队列为
P1P2P3P4P5C1C2C3C4C5C6C7P6P7
 - 规则1：阻塞进程被唤醒后立即插入队列头作为下一个执行对象
 - 规则2：阻塞进程被唤醒后将放入队列末尾
 - 提示：阻塞进程被唤醒后将从阻塞的地方往下执行，不再执行前面的语句。例如因为wait操作而引发的阻塞，被唤醒后，wait操作不会在执行，而是执行wait操作后面的语句。
- 初始化**mutex=1, empty=3, full=0**
- 若遵循规则1，请描述队列的执行过程和变量**empty**、**full**到变化
- 若遵循规则2，请描述队列的执行过程和变量**empty**、**full**到变化

2.4 经典进程的同步问题

规则1的执行结果

执行	执行后	信号量变化	剩余队列	empty阻塞	full阻塞
P1	P1完成	empty=2, full=1	P2P3P4P5C1C2C3C4C5C6C7P6P7		
P2	P2完成	empty=1, full=2	P3P4P5C1C2C3C4C5C6C7P6P7		
P3	P3完成	empty=0, full=3	P4P5C1C2C3C4C5C6C7P6P7		
P4	P4阻塞	empty=-1, full=3	P5C1C2C3C4C5C6C7P6P7	P4	
P5	P5阻塞	empty=-2, full=3	C1C2C3C4C5C6C7P6P7	P4, P5	
C1	C1完成, 唤醒P4	empty=-1, full=2	P4C2C3C4C5C6C7P6P7	P5	
P4	P4完成	empty=-1, full=3	C2C3C4C5C6C7P6P7	P5	
C2	C2完成, 唤醒P5	empty=0, full=2	P5C3C4C5C6C7P6P7		
P5	P5完成	empty=0, full=3	C3C4C5C6C7P6P7		
C3	C3完成	empty=1, full=2	C4C5C6C7P6P7		
C4	C4完成	empty=2, full=1	C5C6C7P6P7		
C5	C5完成	empty=3, full=0	C6C7P6P7		
C6	C6阻塞	empty=3, full=-1	C7P6P7		C6
C7	C7阻塞	empty=3, full=-2	P6P7		C6, C7
P6	P6完成, 唤醒C6	empty=2, full=-1	C6P7		C7
C6	C6完成	empty=3, full=-1	P7		C7
P7	P7完成, 唤醒C7	empty=2, full=0	C7		
C7	C7完成	empty=3, full=0	空		

遵循规则2的执行结果

执行	执行后	信号量变化	剩余队列	empty阻塞	full阻塞
P1	P1完成	empty=2, full=1	P2P3P4P5C1C2C3C4C5C6C7P6P7		
P2	P2完成	empty=1, full=2	P3P4P5C1C2C3C4C5C6C7P6P7		
P3	P3完成	empty=0, full=3	P4P5C1C2C3C4C5C6C7P6P7		
P4	P4阻塞	empty=-1, full=3	P5C1C2C3C4C5C6C7P6P7	P4	
P5	P5阻塞	empty=-2, full=3	C1C2C3C4C5C6C7P6P7	P4, P5	
C1	C1完成, 唤醒P4	empty=-1, full=2	C2C3C4C5C6C7P6P7P4	P5	
C2	C2完成, 唤醒P5	empty=0, full=1	C3C4C5C6C7P6P7P4P5		
C3	C3完成	empty=1, full=0	C4C5C6C7P6P7P4P5		
C4	C4阻塞	empty=1, full=-1	C5C6C7P6P7P4P5		C4
C5	C5阻塞	empty=1, full=-2	C6C7P6P7P4P5		C4, C5
C6	C6阻塞	empty=1, full=-3	C7P6P7P4P5		C4, C5, C6
C7	C7阻塞	empty=1, full=-4	P6P7P4P5		C4, C5, C6, C7
P6	P6完成, 唤醒C4	empty=0, full=-3	P7P4P5C4		C5, C6, C7
P7	P7阻塞	empty=-1, full=-3	P4P5C4	P7	C5, C6, C7
P4	P4阻塞	empty=-2, full=-3	P5C4	P7, P4	C5, C6, C7
P5	P5阻塞	empty=-3, full=-3	C4	P7, P4, P5	C5, C6, C7
C4	C4完成, 唤醒P7	empty=-2, full=-3	P7	P4, P5	C5, C6, C7
P7	P7完成, 唤醒C5	empty=-2, full=-2	C5	P4, P5	C6, C7
C5	C5完成, 唤醒P4	empty=-1, full=-2	P4	P5	C6, C7
P4	P4完成, 唤醒C6	empty=-1, full=-1	C6	P5	C7
C6	C6完成, 唤醒P5	empty=0, full=-1	P5		C7
P5	P5完成, 唤醒C7	empty=0, full=0	C7		
C7	C7完成, 结束	empty=1, full=0	空		



遵循规则2的执行结果

执行	执行后	信号量变化	剩余队列	empty阻塞	full阻塞
P1	P1完成	empty=2, full=1	P2P3P4P5C1C2C3C4C5C6C7P6P7		
P2	P2完成	empty=1, full=2	P3P4P5C1C2C3C4C5C6C7P6P7		
P3	P3完成	empty=0, full=3	P4P5C1C2C3C4C5C6C7P6P7		
P4	P4阻塞	empty=-1, full=3	P5C1C2C3C4C5C6C7P6P7	P4	
P5	P5阻塞	empty=-2, full=3	C1C2C3C4C5C6C7P6P7	P4, P5	
C1	C1完成, 唤醒P4	empty=-1, full=2	C2C3C4C5C6C7P6P7P4	P5	
C2	C2完成, 唤醒P5	empty=0, full=1	C3C4C5C6C7P6P7P4P5		
C3	C3完成	empty=1, full=0	C4C5C6C7P6P7P4P5		
C4	C4阻塞	empty=1, full=-1	C5C6C7P6P7P4P5		C4
C5	C5阻塞	empty=1, full=-2	C6C7P6P7P4P5		C4, C5
C6	C6阻塞	empty=1, full=-3	C7P6P7P4P5		C4, C5, C6
C7	C7阻塞	empty=1, full=-4	P6P7P4P5		C4, C5, C6, C7
P6	P6完成, 唤醒C4	empty=0, full=-3	P7P4P5C4		C5, C6, C7
P7	P7阻塞	empty=-1, full=-3	P4P5C4	P7	C5, C6, C7
P4	P4完成, 唤醒C5	empty=-1, full=-2	P5C4C5	P7	C6, C7
P5	P5完成, 唤醒C6	empty=-1, full=-1	C4C5C6	P7	C7
C4	C4完成, 唤醒P7	empty=0, full=-1	C5C6P7		C7
C5	C5完成	empty=1, full=-1	C6P7		C7
C6	C6完成	empty=2, full=-1	P7		C7
P7	P7完成, 唤醒C7	empty=2, full=0	C7		
C7	C7完成	empty=3, full=0	空		

2.4 经典进程的同步问题

- 遵循规则1运行18次结束
- 遵循规则2运行21次结束
- 规则1发生的阻塞比规则2少，执行的步骤也少
- 因此，阻塞的进程被唤醒后应该优先执行，这样可以减少后续发生阻塞的次数。

2.4 经典进程的同步问题

2. 利用AND信号量解决生产者—消费者问题

```
Var mutex, empty, full:semaphore  : =  1, n, 0;  
    buffer:array [0, ..., n-1] of item;  
    in out:integer    : =  0, 0;  
begin  
    parbegin  
        producer:begin  
            repeat  
                produce an item in nextp;  
                Swait(empty, mutex);  
                buffer(in) : =  nextp;  
                in  : =  (in+1)mod n;  
                Ssignal(mutex, full);  
            until false;  
        end
```

2.4 经典进程的同步问题

■ 续前程序

```
consumer:begin
  repeat
    Swait(full, mutex);
    nextc  : =  buffer(out);
    out   : =  (out+1) mod n;
    Ssignal(mutex, empty);
    consumer the item in nextc;
  until false;
end
parend
end
```

2.4 经典进程的同步问题

■ 管程解决生产者-消费者问题

producer:begin

repeat

produce an item in nextp;

PC.put(item);

until false;

end

consumer:begin

repeat

PC.get(item);

consume the item in nextc;

until false;

end

练习：公交车问题

- 在公共汽车上有司机和售票员
- 司机的活动：
 - ☐ 启动车辆
 - ☐ 正常运行
 - ☐ 到站停车
- 售票员的活动
 - ☐ 上下乘客
 - ☐ 关车门
 - ☐ 售票
 - ☐ 开车门
 - ☐ 上下乘客
- 条件1：司机必须等售票员关车门后才能启动车辆
- 条件2：售票员必须等司机停车后才能开车门
- 请设计程序，使用记录型信号量实现两人的同步合作

练习：公交车问题

- 司机和售票员两个角色
 - 两个进程driver和busman
- 条件1：关车门(b)→启动车辆(d)
 - 设置信号量S1对应司机能否启动车辆
 - S1的检查在司机进程，开锁在售票员进程
- 条件2：到站停车(d)→开车门(b)
 - 设置信号量S2对应售票员能否开车门
 - S2的检查在售票员进程，开锁在司机进程

```
int S1=0, S2=0;
main(){
    driver();
    busman();
}
```

//S1能否初始为1?

```
driver() {
    while(1)
    { wait(S1);
      启动车辆;
      正常运行;
      到站停车;
      signal(S2);
    }
}
```

```
busman() {
    while(1)
    {上下乘客;
      关车门;
      signal(S1);
      售票;
      wait(S2);
      开车门;
      上下乘客; }
}
```

练习

- 仓库管理有入货和出货两个操作。仓库内共有**50**个空位，最多存放**50**件货物。另外有一张登记表，无论入货或出货都需要在上面做登记。

- 入货流程是：仓库登记，存放货物

- 出货流程是：仓库登记，取出货物

Semaphore tmutex = 1, shelf = 50, item=0;

In() {

_____ ; //信号量操作

_____ ; //信号量操作

登记;

存放货物;

_____ ; //信号量操作

_____ ; //信号量操作

}

Out() {

_____ ; //信号量操作

_____ ; //信号量操作

登记;

取出货物;

_____ ; //信号量操作

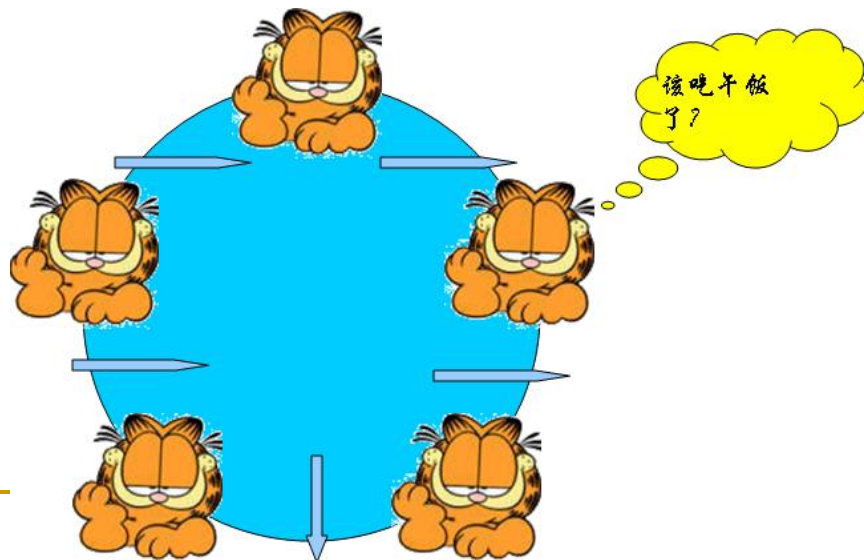
_____ ; //信号量操作

}

2.4 经典进程的同步问题

2.4.2 哲学家进餐问题

- 五个哲学家围坐一张圆桌
- 每两个哲学家之间有一根筷子
- 每个哲学家只能拿相邻的两根筷子，每次只能拿一根
- 每个哲学家只有拿到两根筷子才能进食
- 问题：五个哲学家都去拿筷子，如何用信号量描述他们的活动？能否避免死锁，使得至少一位哲学家能就餐？



2.4 经典进程的同步问题

1. 利用记录型信号量解决哲学家进餐问题

- 筷子是临界资源，在一段时间内只允许一位哲学家使用
- 实现对筷子的互斥使用，每个信号量对应一只筷子
- 五个信号量构成信号量数组：
 - **Var chopstick: array [0, ..., 4] of semaphore;**
 - 所有信号量均被初始化为1

2.4 经典进程的同步问题

- 每位哲学家的程序:

repeat

wait(chopstick [i]);

wait(chopstick [(i+1) mod 5]);

...

eat;

...

signal(chopstick [i]);

signal(chopstick [(i+1) mod 5]);

...

- 解法保证不会有两个相邻的哲学家同时进餐
- 仍然有可能发生死锁
 - 五个哲学家同时拿起左边的筷子，使五个信号量均为0
 - 当他们再试图去拿右边的筷子，都将因无筷子而死锁

2.4 经典进程的同步问题

■ 避免死锁的方法：

- ① 至多只允许有四位哲学家同时去拿左边的筷子
- ② 当哲学家的左、右两只筷子均可用时，才允许他拿起筷子进餐。
- ③ 规定奇数号哲学家先拿他左边的筷子，然后再去拿右边的筷子；而偶数号哲学家则相反。

□ 1、2号哲学家竞争1号筷子；3、4号哲学家竞争3号筷子。即五位哲学家都先竞争奇数号筷子，获得后，再去竞争偶数号筷子，最后总会有一位哲学家能获得两只筷子而进餐。

2.4 经典进程的同步问题

2.4.3 读者-写者问题

- 一个数据文件可被多个进程共享
- 允许多个进程同时读，禁止多个进程同时写
- 当一个进程写的时候，其他所有读进程都要停止
- 当进程读的时候，不允许写进程的发生
- 读写是互斥的

2.4 经典进程的同步问题

1. 利用记录型信号量解决读者-写者问题

- 互斥信号量**Wmutex**: 实现**Reader**与**Writer**进程的互斥
- 整型变量**Readcount**: 正在读的进程数目
 - 当**Readcount** > 0 , 说明现在已有进程在读数据, 没有写进程, 所以其他读进程可以立即参与读操作, 无需上锁
 - 当**Readcount** $= 0$, 说明现在无读进程, 可能有写进程, 这时候读进程要上锁**wait(Wmutex)**排斥写进程
 - 当**Readcount** $= 0$ 且**wait(Wmutex)**上锁成功, 读进程才能往下执行, 并执行**Readcount+1**
 - 每个读进程退出时执行**Readcount-1**, 然后若**Readcount** $= 0$, 说明无读进程, 开锁让写进程进来。
- 因为**Readcount**是一个可被多个**Reader**进程访问的临界资源, 因此, 应该为它设置一个互斥信号量**rmutex**

2.4 经典进程的同步问题

■ 读者-写者问题的记录型信号量程序

```
Var rmutex, wmutex:semaphore    : =  1,1;
  Readcount:integer    : =  0;
begin
  parbegin
    READER:begin
      repeat
        wait(rmutex);
        if readcount=0 then wait(wmutex);
        readcount: =  readcount+1;
        signal(rmutex);
        perform read operation;
        ...
```

■ 不考虑readcount<0的情况，因为程序出错时才会发生

2.4 经典进程的同步问题

■ 续前程序

```
wait(rmutex);  
readcount  : =  readcount-1;  
if readcount=0 then signal(wmutex);  
signal(rmutex);  
until false;  
end
```

```
WRITER:begin  
  repeat  
    wait(wmutex);  
    perform write operation;  
    signal(wmutex);  
  until false;  
  ...
```

2.4 经典进程的同步问题

2. 利用信号量集机制解决读者-写者问题

- 增加限制，最多允许**RN**个读者读
- 信号量**L**，初值**RN**，用于控制读者数目
 - 在第**RN+1**个读者要进入读时，会引起阻塞
- 使用**Swait(S, 1, 0)**的方法实现开关作用

2.4 经典进程的同步问题

- 读者-写者问题的信号量集程序

```
Var RN integer;  
    L, mx:semaphore    : =    RN,1;  
begin  
    parbegin  
        READER:begin  
            repeat  
                Swait(L,1,1);  
                Swait(mx,1,0);  
  
                ...  
                perform read operation;  
  
                ...  
                Ssignal(L,1);  
            until false;  
        end
```

2.4 经典进程的同步问题

```
WRITER:begin
  repeat
    Swait(mx,1,1; L,RN,0);
    perform write operation;
    Ssignal(mx,1);
  until false;
end
parend
end
```